

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. Шухова»
(БГТУ им. В. Г. Шухова)



ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И УПРАВЛЯЮЩИХ СИСТЕМ

Кафедра программного обеспечения вычислительной
техники и автоматизированных систем

Лабораторная работа №1

по дисциплине: «Тестирование ПС»

по теме: «Составление тест-кейсов для готового кода.
Качество ПО и место тестирования в жизненном цикле»

Выполнил: студент группы ПВ-233

Алексеев Андрей Евгеньевич

Проверил: преподаватель

Доденко Олег Павлович

Белгород, 2026 г.

Цель работы: сформировать системное представление о качестве ПО и роли тестирования в жизненном цикле; приобрести практические навыки проектирования тест-кейсов и написания автотестов на основе готового кода.

Содержание

Задание 1.....	3
loyalty_points.py.....	3
Бизнес-правила.....	4
Входные данные	4
Выходные данные	4
Допустимые значения.....	5
Недопустимые значения.....	5
Граничные значения.....	5
Задание 2.....	6
Позитивные ТС.....	6
Негативные ТС	6
Граничные ТС.....	6
ТС, покрывающие классы эквивалентности	7
Задание 3.....	8
Задание 4.....	10
test_loyalty_points.py	10
Задание 5.....	13
Результат работы программы.....	13
Вывод.....	13

Задания

1. Проанализировать готовый модуль и выделить бизнес-правила, входные и выходные данные, ограничения.
2. Составить набор тест-кейсов: позитивных, негативных, граничных, покрывающих эквивалентные классы.
3. Оформить тест-кейсы по стандартным атрибутам.
4. Реализовать автотесты на Python + pytest.
5. Запустить тесты и зафиксировать результаты.

Задание 1. Проанализировать готовый модуль и выделить бизнес-правила, входные и выходные данные, ограничения.

loyalty_points.py:

```
# Расчёт бонусных баллов лояльности за аренду автомобиля
def calculate_loyalty_points(
    rental_days: int,
    car_class: str,
    is_regular_customer: bool,
) -> int:

    if rental_days <= 0:
        raise ValueError("rental_days must be > 0")

    base_rates = {
        "economy": 10,
        "comfort": 20,
        "business": 40,
    }
    if car_class not in base_rates:
        raise ValueError(
            f"car_class must be one of {list(base_rates.keys())}, got {car_class!r}"
        )

    points = base_rates[car_class] * rental_days

    if rental_days <= 3:
        duration_multiplier = 1.0
    elif rental_days <= 7:
        duration_multiplier = 1.1
    elif rental_days <= 29:
        duration_multiplier = 1.25
    else:
        duration_multiplier = 1.5

    points *= duration_multiplier

    if is_regular_customer:
        points *= 1.5

    return round(points)
```

Данная функция рассчитывает количество бонусных баллов лояльности, которые начисляются клиенту за аренду автомобиля. Расчёт зависит от: длительности аренды, класса автомобиля, статуса постоянного клиента. Результат — целое число (баллы), возвращается через `round()`.

Модуль реализует чистую детерминированную функцию без побочных эффектов и внешних зависимостей. Логика линейна: валидация → базовая ставка → множитель за длительность → множитель постоянного клиента → округление.

Бизнес-правила:

№	Правило	Условие в коде	Результат
1	Длительность аренды должна быть положительной	<code>rental_days <= 0</code>	<code>ValueError("rental_days must be > 0")</code>
2	Класс автомобиля — только из фиксированного списка	<code>car_class not in {"economy", "comfort", "business"}</code>	<code>ValueError("car_class must be one of ...")</code>
3	Базовая ставка баллов за сутки зависит от класса	<code>economy → 10, comfort → 20, business → 40</code>	баллов/сутки
4	Множитель за длительность аренды (поощрение долгосрочной аренды)	<code>≤3 дн → ×1.0, 4–7 дн → ×1.1, 8–29 дн → ×1.25, ≥30 дн → ×1.5</code>	<code>points *= duration_multiplier</code>
5	Постоянный клиент получает повышенный коэффициент	<code>is_regular_customer=True</code>	<code>points *= 1.5</code>
6	Результат округляется до целого	<code>return round(points)</code>	int

Входные данные:

Параметр	Тип	Описание	Ограничения
<code>rental_days</code>	int	Количество дней аренды	<code>> 0</code>
<code>car_class</code>	str	Класс автомобиля	∈ {"economy", "comfort", "business"}
<code>is_regular_customer</code>	bool	Постоянный клиент	True / False

Выходные данные:

Параметр	Тип	Описание
(return)	int	Количество баллов

Допустимые значения:

Параметр	Допустимые значения
rental_days	Любое целое ≥ 1 (1, 2, 3, ..., ∞)
car_class	"economy", "comfort", "business"
is_regular_customer	True, False

Недопустимые значения:

Параметр	Недопустимые значения	Что произойдёт
rental_days	0, отрицательные	ValueError
car_class	Любая строка вне списка: "premium", "Economy", "", "sport", None	ValueError

Граничные значения:

Параметр	Границы	Почему важны
rental_days	0 / 1	Нижняя граница валидации
rental_days	3 / 4, 7 / 8, 29 / 30	Переключение duration_multiplier: $\times 1.0 \rightarrow \times 1.1 \rightarrow \times 1.25 \rightarrow \times 1.5$
car_class	три значения	Разные базовые ставки
is_regular_customer	True / False	Переключение множителя 1.5
Итоговый балл	.5 после round()	Округление до четного

Задание 2. Составить набор тест-кейсов: позитивных, негативных, граничных, покрывающих эквивалентные классы.

Позитивные ТС:

ID	Название	Вход	Ожидание
ТС-01	1 день, economy, не постоянный	(1, "economy", False)	10
ТС-02	1 день, comfort, не постоянный	(1, "comfort", False)	20
ТС-03	1 день, business, не постоянный	(1, "business", False)	40
ТС-04	5 дней, comfort, постоянный	(5, "comfort", True)	165
ТС-05	10 дней, business, постоянный	(10, "business", True)	750
ТС-06	60 дней, economy, не постоянный	(60, "economy", False)	900

Негативные ТС:

ID	Название	Вход	Ожидание
ТС-07	Ноль дней	(0, "economy", False)	ValueError
ТС-08	Отрицательные дни	(-5, "economy", False)	ValueError
ТС-09	Неизвестный класс	(3, "premium", False)	ValueError
ТС-10	Класс в другом регистре	(3, "Economy", False)	ValueError

Граничные ТС:

ID	Название	Вход	Ожидание
ТС-11	3 дня (граница $\times 1.0 \rightarrow \times 1.1$)	(3, "economy", False)	30
ТС-12	4 дня (граница $\times 1.0 \rightarrow \times 1.1$)	(4, "economy", False)	44
ТС-13	7 дней (граница $\times 1.1 \rightarrow \times 1.25$)	(7, "comfort", False)	154
ТС-14	8 дней (граница $\times 1.1 \rightarrow \times 1.25$)	(8, "comfort", False)	200
ТС-15	29 дней (граница $\times 1.25 \rightarrow \times 1.5$)	(29, "business", False)	1450
ТС-16	30 дней (граница $\times 1.25 \rightarrow \times 1.5$)	(30, "business", False)	1800

ТС, покрывающие классы эквивалентности:

ID	Класс	Вход	Ожидание
ТС-17	E1: $\text{days} \leq 0$	(-1, "economy", False)	ValueError
ТС-18	E2: невалидный класс	(3, "sport", False)	ValueError
ТС-19	E3: 1–3 дня, $\times 1.0$	(2, "economy", False)	20
ТС-20	E4: 4–7 дней, $\times 1.1$	(5, "economy", False)	55
ТС-21	E5: 8–29 дней, $\times 1.25$	(15, "economy", False)	188
ТС-22	E6: ≥ 30 дней, $\times 1.5$	(30, "economy", False)	450
ТС-23	E7: постоянный клиент, $\times 1.5$	(2, "economy", True)	30
ТС-24	E8: не постоянный клиент	(2, "economy", False)	20
ТС-25	E9: три класса	(1, "comfort", False)	20

Задание 3. Оформить тест-кейсы по стандартным атрибутам.

ТС-26. 2 дня, business, постоянный

Атрибут	Значение
ID	ТС-26
Название	2 дня, business, постоянный клиент
Приоритет	High
Предусловие	Модуль loyalty_points импортирован
Шаги	1. Вызвать calculate_loyalty_points(2, "business", True)
Тестовые данные	rental_days=2, car_class="business", is_regular_customer=True
Ожидаемый результат	Возвращается 120
Постусловие	—
Тип	Позитивный
Класс эквивалентности	E3, E7, E9

ТС-27. 15 дней, comfort, постоянный

Атрибут	Значение
ID	ТС-27
Название	15 дней, comfort, постоянный клиент
Приоритет	High
Предусловие	Модуль loyalty_points импортирован
Шаги	1. Вызвать calculate_loyalty_points(15, "comfort", True)
Тестовые данные	rental_days=15, car_class="comfort", is_regular_customer=True
Ожидаемый результат	Возвращается 562
Постусловие	—
Тип	Позитивный
Класс эквивалентности	E5, E7, E9

ТС-28. 45 дней, business, не постоянный

Атрибут	Значение
ID	ТС-28
Название	45 дней, business, не постоянный клиент
Приоритет	Medium
Предусловие	Модуль loyalty_points импортирован
Шаги	1. Вызвать calculate_loyalty_points(45, "business", False)
Тестовые данные	rental_days=45, car_class="business", is_regular_customer=False
Ожидаемый результат	Возвращается 2700
Постусловие	—
Тип	Позитивный
Класс эквивалентности	E6, E8, E9

ТС-29. 0.5 дня

Атрибут	Значение
ID	ТС-29
Название	1 день, business, постоянный клиент
Приоритет	High
Предусловие	Модуль loyalty_points импортирован
Шаги	1. Вызвать calculate_loyalty_points(1, "business", True)
Тестовые данные	rental_days=1, car_class="business", is_regular_customer=True
Ожидаемый результат	Возвращается 60
Постусловие	—
Тип	Позитивный (граница нижняя)
Класс эквивалентности	E3, E7, E9

ТС-30. 100 дней, еconomy, постоянный

Атрибут	Значение
ID	ТС-30
Название	100 дней, еconomy, постоянный клиент
Приоритет	Medium
Предусловие	Модуль loyalty_points импортирован
Шаги	1. Вызвать calculate_loyalty_points(100, "economy", True)
Тестовые данные	rental_days=100, car_class="economy", is_regular_customer=True
Ожидаемый результат	Возвращается 2250
Постусловие	—
Тип	Позитивный
Класс эквивалентности	E6, E7, E9

Задание 4. Реализовать автотесты на Python + pytest.

test_loyalty_points.py:

```
import pytest
from loyalty_points import calculate_loyalty_points

class TestPositive:
    """Позитивные сценарии"""

    def test_tc01_economy_1_day(self):
        """TC-01: 1 день, economy, не постоянный."""
        assert calculate_loyalty_points(1, "economy", False) == 10

    def test_tc02_comfort_1_day(self):
        """TC-02: 1 день, comfort, не постоянный."""
        assert calculate_loyalty_points(1, "comfort", False) == 20

    def test_tc03_business_1_day(self):
        """TC-03: 1 день, business, не постоянный."""
        assert calculate_loyalty_points(1, "business", False) == 40

    def test_tc04_comfort_5_days_regular(self):
        """TC-04: 5 дней, comfort, постоянный."""
        assert calculate_loyalty_points(5, "comfort", True) == 165

    def test_tc05_business_10_days_regular(self):
        """TC-05: 10 дней, business, постоянный."""
        assert calculate_loyalty_points(10, "business", True) == 750

    def test_tc06_economy_60_days(self):
        """TC-06: 60 дней, economy, не постоянный."""
        assert calculate_loyalty_points(60, "economy", False) == 900

class TestNegative:
    """Негативные сценарии"""

    def test_tc07_zero_days(self):
        """TC-07: ноль дней → ValueError."""
        with pytest.raises(ValueError, match="rental_days"):
            calculate_loyalty_points(0, "economy", False)

    def test_tc08_negative_days(self):
        """TC-08: отрицательные дни → ValueError."""
        with pytest.raises(ValueError, match="rental_days"):
            calculate_loyalty_points(-5, "economy", False)

    def test_tc09_unknown_class(self):
        """TC-09: неизвестный класс → ValueError."""
        with pytest.raises(ValueError, match="car_class"):
            calculate_loyalty_points(3, "premium", False)

    def test_tc10_wrong_case_class(self):
        """TC-10: класс в другом регистре → ValueError."""
        with pytest.raises(ValueError, match="car_class"):
            calculate_loyalty_points(3, "Economy", False)

class TestBoundary:
    """Граничные значения"""
```

```

def test_tc11_boundary_3_days(self):
    """TC-11: 3 дня – множитель ×1.0."""
    assert calculate_loyalty_points(3, "economy", False) == 30

def test_tc12_boundary_4_days(self):
    """TC-12: 4 дня – множитель ×1.1."""
    assert calculate_loyalty_points(4, "economy", False) == 44

def test_tc13_boundary_7_days(self):
    """TC-13: 7 дней – множитель ×1.1."""
    assert calculate_loyalty_points(7, "comfort", False) == 154

def test_tc14_boundary_8_days(self):
    """TC-14: 8 дней – множитель ×1.25."""
    assert calculate_loyalty_points(8, "comfort", False) == 200

def test_tc15_boundary_29_days(self):
    """TC-15: 29 дней – множитель ×1.25."""
    assert calculate_loyalty_points(29, "business", False) == 1450

def test_tc16_boundary_30_days(self):
    """TC-16: 30 дней – множитель ×1.5."""
    assert calculate_loyalty_points(30, "business", False) == 1800

```

```

class TestEquivalenceClasses:

```

```

    """Проверка каждого класса эквивалентности"""

    def test_tc17_e1_days_le_zero(self):
        """TC-17 (E1): days ≤ 0 → ValueError."""
        with pytest.raises(ValueError):
            calculate_loyalty_points(-1, "economy", False)

    def test_tc18_e2_invalid_class(self):
        """TC-18 (E2): невалидный класс → ValueError."""
        with pytest.raises(ValueError):
            calculate_loyalty_points(3, "sport", False)

    def test_tc19_e3_short_rental(self):
        """TC-19 (E3): 1–3 дня, ×1.0."""
        assert calculate_loyalty_points(2, "economy", False) == 20

    def test_tc20_e4_week_rental(self):
        """TC-20 (E4): 4–7 дней, ×1.1."""
        assert calculate_loyalty_points(5, "economy", False) == 55

    def test_tc21_e5_middle_rental(self):
        """TC-21 (E5): 8–29 дней, ×1.25."""
        assert calculate_loyalty_points(15, "economy", False) == 188

    def test_tc22_e6_long_rental(self):
        """TC-22 (E6): ≥30 дней, ×1.5."""
        assert calculate_loyalty_points(30, "economy", False) == 450

    def test_tc23_e7_regular_customer(self):
        """TC-23 (E7): постоянный клиент, ×1.5."""
        assert calculate_loyalty_points(2, "economy", True) == 30

    def test_tc24_e8_regular_customer_false(self):
        """TC-24 (E8): обычный клиент, без множителя."""
        assert calculate_loyalty_points(2, "economy", False) == 20

```

```
def test_tc25_e9_comfort_class(self):
    """TC-25 (E9): comfort-класс."""
    assert calculate_loyalty_points(1, "comfort", False) == 20

class TestAdditional:
    """Дополнительные сценарии"""

    def test_tc26_business_2_days_regular(self):
        """TC-26: 2 дня, business, постоянный."""
        assert calculate_loyalty_points(2, "business", True) == 120

    def test_tc27_rounding_half_to_even(self):
        """TC-27: 15 дней, comfort, постоянный → round(562.5) = 562."""
        assert calculate_loyalty_points(15, "comfort", True) == 562

    def test_tc28_business_45_days(self):
        """TC-28: 45 дней, business, не постоянный."""
        assert calculate_loyalty_points(45, "business", False) == 2700

    def test_tc29_business_1_day_regular(self):
        """TC-29: 1 день, business, постоянный (нижняя граница + флаг)."""
        assert calculate_loyalty_points(1, "business", True) == 60

    def test_tc30_economy_100_days_regular(self):
        """TC-30: 100 дней, економу, постоянный."""
        assert calculate_loyalty_points(100, "economy", True) == 2250
```

Задание 5. Запустить тесты и зафиксировать результаты.

Результат работы программы:

```
tests/test_loyalty_points.py::TestPositive::test_tc01_economy_1_day PASSED [ 3%]
tests/test_loyalty_points.py::TestPositive::test_tc02_comfort_1_day PASSED [ 6%]
tests/test_loyalty_points.py::TestPositive::test_tc03_business_1_day PASSED [ 10%]
tests/test_loyalty_points.py::TestPositive::test_tc04_comfort_5_days_regular PASSED [ 13%]
tests/test_loyalty_points.py::TestPositive::test_tc05_business_10_days_regular PASSED [ 16%]
tests/test_loyalty_points.py::TestPositive::test_tc06_economy_60_days PASSED [ 20%]
tests/test_loyalty_points.py::TestNegative::test_tc07_zero_days PASSED [ 23%]
tests/test_loyalty_points.py::TestNegative::test_tc08_negative_days PASSED [ 26%]
tests/test_loyalty_points.py::TestNegative::test_tc09_unknown_class PASSED [ 30%]
tests/test_loyalty_points.py::TestNegative::test_tc10_wrong_case_class PASSED [ 33%]
tests/test_loyalty_points.py::TestBoundary::test_tc11_boundary_3_days PASSED [ 36%]
tests/test_loyalty_points.py::TestBoundary::test_tc12_boundary_4_days PASSED [ 40%]
tests/test_loyalty_points.py::TestBoundary::test_tc13_boundary_7_days PASSED [ 43%]
tests/test_loyalty_points.py::TestBoundary::test_tc14_boundary_8_days PASSED [ 46%]
tests/test_loyalty_points.py::TestBoundary::test_tc15_boundary_29_days PASSED [ 50%]
tests/test_loyalty_points.py::TestBoundary::test_tc16_boundary_30_days PASSED [ 53%]
tests/test_loyalty_points.py::TestEquivalenceClasses::test_tc17_e1_days_le_zero PASSED [ 56%]
tests/test_loyalty_points.py::TestEquivalenceClasses::test_tc18_e2_invalid_class PASSED [ 60%]
tests/test_loyalty_points.py::TestEquivalenceClasses::test_tc19_e3_short_rental PASSED [ 63%]
tests/test_loyalty_points.py::TestEquivalenceClasses::test_tc20_e4_week_rental PASSED [ 66%]
tests/test_loyalty_points.py::TestEquivalenceClasses::test_tc21_e5_middle_rental PASSED [ 70%]
tests/test_loyalty_points.py::TestEquivalenceClasses::test_tc22_e6_long_rental PASSED [ 73%]
tests/test_loyalty_points.py::TestEquivalenceClasses::test_tc23_e7_regular_customer PASSED [ 76%]
tests/test_loyalty_points.py::TestEquivalenceClasses::test_tc24_e8_regular_customer_false PASSED [ 80%]
tests/test_loyalty_points.py::TestEquivalenceClasses::test_tc25_e9_comfort_class PASSED [ 83%]
tests/test_loyalty_points.py::TestAdditional::test_tc26_business_2_days_regular PASSED [ 86%]
tests/test_loyalty_points.py::TestAdditional::test_tc27_rounding_half_to_even PASSED [ 90%]
tests/test_loyalty_points.py::TestAdditional::test_tc28_business_45_days PASSED [ 93%]
tests/test_loyalty_points.py::TestAdditional::test_tc29_business_1_day_regular PASSED [ 96%]
tests/test_loyalty_points.py::TestAdditional::test_tc30_economy_100_days_regular PASSED [100%]

===== 30 passed in 0.20s =====
```

Вывод: выполняя лабораторную работу, я сформировал системное представление о качестве ПО и роли тестирования в жизненном цикле, а также приобрел практические навыки проектирования тест-кейсов и написания автотестов на основе готового кода. Проанализировал функцию `calculate_loyalty_points`, выделил её бизнес-правила и граничные значения, составил 30 тест-кейсов (позитивных, негативных, граничных и покрывающих классы эквивалентности) и реализовал их в виде автотестов на `pytest`.